



Синтез и анализ на алгоритми за факултети ФКСТ, ФТК, ФЕТТ

проф. д-р Даниела Минковска
каб. 2324, кат. ПКТ, ФКСТ, ТУ-София
daniela@tu-sofia.bg



Лекция 11

Алгоритми за аритметични действия.

Бройни системи, преобразуване на числата. Прав, обратен и допълнителен код. Двоично-десетични кодове. Коректност (верификация) на алгоритъм.



Принцип на програмното управление

Основен принцип: **Работата на КС се управлява от програма.**

- **Програма** - последователност от действия, които се извършват от КС, при обработка на входните данни с цел да се получат изходни данни (резултати).
- **Системни програми** - служат за управление на ресурсите на компютъра:
 - операционни системи
 - драйвери и др.
- **Приложните програми** – създават се за обслужване на потребителите.
- **Данни** - числа и символи, представят се като последователности от 0 и 1 с определен размер. Примери за знакови кодове – ASCII, windows-1252, UTF-8, UTF-16 и др.



Бройни системи

Бройна система - символичен метод за представяне на числата посредством ограничен брой символи, наречени цифри, т.е съвкупност от **начини и правила за изобразяване на числата**.

Съществуват два вида бройни системи:

-Непозиционни - стойността на цифрата не зависи от нейното място в записването на числото. *Недостатък*: трудност за запис на големите числа. Представянето на големите числа изисква много дълъг запис или много голям набор от символи за означаване на възлови количества.

-Позиционни - стойността на всяка цифра се определя от нейното място в записването на числото.



Таблица на използваните позиционни бройни системи

Десетична система	Двоична система	Осмична система	Шестнадесетична система
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F



Бройни системи

Преобразуване на число от коя да е БС с основа S в БС с основа 10:

Всяко число в произволна позиционна бройна система може да се представи по следния начин:

$$A = a_n S^n + a_{n-1} S^{n-1} + \dots + a_1 S^1 + a_0 S^0 + a_{-1} S^{-1} + \dots = \sum_{i=-\infty}^{\infty} a_i S^i$$

където:

A – число в позиционна бройна система;

a – цифра на бройната система;

S – основа на системата;

S^i – тегло на позицията с номер i ;

i – позиция (разряд) в записа на числото;



Бройни системи

Примери за преобразуване на число от коя да е БС с основа S в БС с основа 10 :

Пример1: Да се преобразува двоичното число **111001** в десетична БС:

$$111001 = 1.2^5 + 1.2^4 + 1.2^3 + 0.2^2 + 0.2^1 + 1.2^0 = 32+16+8+0+0+1 = 57$$

следователно $111001_{(2)} = 57_{(10)}$

Пример 2: Да се преобразува осмичното число **141** в десетична БС:

$$141 = 1.8^2 + 4.8^1 + 1.8^0 = 64+32+1 = 97, \text{ следователно } 141_{(8)} = 97_{(10)}$$

Пример 3: Да се преобразува шестнадесетичното число **4A12** в десетична БС:

$$4A12 = 4.16^3 + 10.16^2 + 1.16^1 + 2.16^0 = 16384+2560+16+2 = 18962$$

следователно $4A12_{(16)} = 18962_{(10)}$



Бройни системи

Преобразуване на число от десетична БС в коя да е БС с основа S :

Алгоритъм:

1. установява се най-младшият разряд, т.е. $i=0$;
повтаряй
2. разделя се числото A на основата S , т.е. частното $R=A/S$;
3. остатъкът от делението се присвоява като значение на най-младшия разряд, т.е. $a_i=A-R.S$;
4. числото A присвоява резултата от делението, т.е. $A=R$;
5. установява се следващият разряд, т.е. $i=i+1$;
докато частното от делението $R>0$.



Бройни системи

Примери за преобразуване на число от десетична БС в коя да е БС с основа S:

Пример 1: Да се преобразува десетичното число 326 в двоична бройна система:

i=0	$R=326/2=163$	$a_0 = 326-163.2 = 0$	
i=1	$R=163/2=81$	$a_1 = 163-81.2 = 1$	
i=2	$R=81/2=40$	$a_2 = 81-40.2 = 1$	
i=3	$R=40/2=20$	$a_3 = 40-20.2 = 0$	
i=4	$R=20/2=10$	$a_4 = 20-10.2 = 0$	
i=5	$R=10/2=5$	$a_5 = 10-5.2 = 0$	
i=6	$R=5/2=2$	$a_6 = 5-2.2 = 1$	
i=7	$R=2/2=1$	$a_7 = 2-1.2 = 0$	
i=8	$R=1/2=0$	$a_8 = 1-0.2 = 1$	

следователно: **$326_{(10)} = 101000110_{(2)}$**



Бройни системи

Примери за преобразуване на число от десетична БС в коя да е БС с основа S:

Пример 2: Да се преобразува десетичното число 118 в осмична бройна система:

i=0	$R=118/8=14$	$a_0 = 118-14 \cdot 8 = 6$	↑
i=1	$R=14/8=1$	$a_1 = 14-1 \cdot 8 = 6$	
i=2	$R=1/8=0$	$a_2 = 1-0 \cdot 8 = 1$	

следователно: $118_{(10)} = 166_{(8)}$

Пример 3: Да се преобразува десетичното число 242 в шестнадесетична бройна система:

i=0	$R=242/16=15$	$a_0 = 242-15 \cdot 16 = 2$	↑
i=1	$R=15/16=0$	$a_1 = 15-0 \cdot 16 = 15 \text{ (F)}$	

следователно: $242_{(10)} = \text{F}2_{(16)}$



Бройни системи

Преобразуване на правилни дроби:

Алгоритъм:

а) установява се най-младши разряд, т.е. $i = -1$;

б) *повтаряй*:

- умножава се числото A с основата S , т.е. $P = A.S$;

- цялата част от умножението се присвоява като значение на най-младшия разряд, т.е. $a_i = \text{цяла част } (P)$;

- числото A присвоява дробната част от произведението, т.е.

$A = \text{дробна част } (P)$;

- установява се следващия разряд, т.е. $i = i - 1$;

докато A стане > 0 .



Бройни системи

Примери за преобразуване на правилни дроби:

Алгоритъм:

Пример 1: Да се преобразува десетичната дроб 0,375 в двоична бройна система:

$i=-1$	$P=0,375 \cdot 2=0,75$	$a_{-1} = 0; A = 0,75$	
$i=-2$	$P=0,75 \cdot 2=1,5$	$a_{-2} = 1; A = 0,5$	
$i=-3$	$P=0,5 \cdot 2=1$	$a_{-3} = 1; A = 0$	

следователно: $0,375_{(10)} = 0,011_{(2)}$



Бройни системи. Аритметични действия

Аритметични действия в двоична система:

Тези операции се извършват по същия начин, както обичайната десетична аритметика. По-особено е само формирането на преноса към по-старшия разряд при събиране и заемането от по-старши разряд при изваждане.



Бройни системи. Аритметични действия

Аритметични действия в двоична система. Събиране

1. Събиране: Числата се събират поразредно, като в резултат на събирането на ***i*-тите разреди** на двете събираеми се **получава *i*-тият разред на сумата им.**

а) **Двоично събиране:** Двоичната аритметика е бърза и лесна тъй като събиранията могат да бъдат само **четири вида:**

- $0+0=0$

- $1+0=1$

- $0+1=1$

- $1+1=0$ пренос (carry) 1

Сума (Sum)

Тъй като имаме само две цифри 0 и 1, пренос се случва твърде често!



Бройни системи. Аритметични действия

Аритметични действия в двоична система. Събиране

Пример 1: Да се *съберат трите двоични числа* и да се *направи проверка* чрез *десетично събиране*:

	проверка:
11011	27
+ 10011	+ 19
11000	24
-----	-----
1000110 ₂	70 ₁₀



Бройни системи. Аритметични действия

Аритметични действия в двоична система. Събиране

б) *Осмично събиране*: Тук трябва да се помни, че **пренос 1** към по-старши разряд **се подава**, когато **сумата е по-голяма или равна на 8**.

Пример 1: Да се съберат двете осмични числа и да се направи проверка, чрез десетично събиране.

проверка:

$$\begin{array}{r} 71_8 \\ + 25_8 \\ \hline 116_8 \end{array}$$

$$\begin{array}{r} 57_{10} \\ + 21_{10} \\ \hline 78_{10} \end{array}$$



Бройни системи. Аритметични действия

Аритметични действия в двоична система. Събиране

б) *Шестнадесетично събиране*: Тук трябва да се помни, че **пренос 1** към по-старши разряд **се подава**, когато **сумата е по-голяма или равна на 16**.

Пример 1: Да се съберат двете шестнадесетични числа и да се направи проверка, чрез десетично събиране.

проверка:

$$\begin{array}{r} BC67_{16} \\ + 594A_{16} \\ \hline 115B1_{16} \end{array} \qquad \begin{array}{r} 48231_{10} \\ + 22858_{10} \\ \hline 71089_{10} \end{array}$$

започвайки от най-младшия разряд на числата, т.е. **7** плюс **A = 7 + 10 = 17**, което е **16 + 1**, следователно **сумата**, която ще запишем като резултат **е 1 и пренос 1** ще добавим към следващата колона. В нея имаме **6 + 4 + 1 (пренос) = 11 = B**, следователно **сумата**, като резултат **е B и няма пренос**. В следващата колона: **C + 9 = 12 + 9 = 21 = 16 + 5**, следователно записваме **5** за сума и **пренос 1**. В финалната колона имаме **B + 5 + 1 (пренос) = 11 + 5 + 1 = 17 = 16 + 1**, следователно записваме **1** за сума и **пренос 1**. И така **резултатната сума е 115B1₁₆**



Бройни системи. Аритметични действия

Аритметични действия в двоична система. Изваждане

2. Изваждане: Когато изваждаме 2 двоични числа може да имаме следните случаи:

- $0-0=0$

- $1-0=1$

- $1-1=0$

- $0-1=1$ заем от предишен разряд ($10 - 1=1$)

Заемът има тегло 2.

Когато предишен разряд е **1**, той става **0** и вземането на заем **се спира**.

Когато предишен разряд е **0**, той става **1** и взимането на заем **продължава** до достигане на **1**, **която става 0** и взимането на заем **спира**.



Бройни системи. Аритметични действия

Аритметични действия в двоична система. Изваждане

Пример 1:

$$\begin{array}{r} 110000110 \\ - 101010100 \\ \hline \end{array}$$

000110010

Пример 2:

$$\begin{array}{r} 11101.110 \\ - 1010.011 \\ \hline \end{array}$$

10011.011

Пример 3:

$$\begin{array}{r} 1000000 \\ - 1 \\ \hline \end{array}$$

0111111

В *Пример 3* заемът преминава през всички разряди.



Бройни системи. Аритметични действия

Аритметични действия в двоична система. Умножение

3. Умножение на двоични числа: За умножение на две числа в произволна позиционна система **се прилагат правилата за умножение при десетичната система**, като за улеснение се използва таблицата за умножение в съответната система.

Умножителната двоична таблица не би поставяла проблеми, тъй като тя заема само четири реда:

$$0 \cdot 0 = 0$$

$$0 \cdot 1 = 0$$

$$1 \cdot 0 = 0$$

$$1 \cdot 1 = 1$$



Бройни системи. Аритметични действия

Аритметични действия в двоична система. Умножение

Правила за умножение на числа в двоична система:

1. Умножаването на двете числа започва от най-младшия разред на множителя. Ако този разред е 1, записва се множимото, а ако е 0, се записва един ред от нули. При 1 в следващия разред на множителя се преписва множимото, изместено с един разред вляво, а при 0 се записва ред нули, също изместени с един разред вляво.

2. Умножаването на двете числа се започва от най-старшия разред на множителя. Ако този разред е 1, записва се множимото, а ако е 0, се записва един ред от нули. При 1 в следващия разред на множителя се преписва множимото, изместено с един разред вдясно, а при 0 се записва ред нули, също изместени с един разред вдясно. След това се преминава към следващия разред и т. н.



Бройни системи. Аритметични действия

Аритметични действия в двоична система. Умножение

Пример 1: Да се извърши умножението на двете двоични числа:

$$\begin{array}{r} 1011 \\ \times 1101 \\ \hline 1011 \\ 0000 \\ 1011 \\ 1011 \\ \hline 10001111 \end{array}$$



Бройни системи. Аритметични действия

Аритметични действия в двоична система. Деление

4. Деление на двоични числа: За деление на две числа в двоична система се прилагат правилата за умножение при десетичната система, но по-просто. $Z=X/Y$

За определяне на текущата цифра Z_i на частното се сравнява текущия остатък R и делителят Y : ако $R \geq Y$, тогава $Z_i=0$, т.е. изборът за цифрата на частното е **0 или 1** (докато при десетичната БС изборът за Z_i е много по-голям – 0,1,2,...,9).

Пример 1:

$$111\ 001:110 = 1001$$

$$111:110 = 1 \text{ остатък } 1$$

$$10:110=0$$

$$100:110 = 0$$

$$1001:110=1 \text{ остатък } 11$$



БС. Прав, обратен и допълнителен код

Необходимост:

Предназначението на кодовете е за по-лесна реализация на аритметичните действия в АЛУ на процесорите. С използването на тези кодове изваждането и събирането се извършват с едно и също устройство – **суматор**, като разликата между събирането на положителни и отрицателни числа е, че при отрицателните преди събирането те се **преобразуват** в съответния код.

При разглеждането на кодовете ще вземем предвид главно дробни числа с фиксирана запетая!



БС. Прав, обратен и допълнителен код

Прав код:

Правият код е най-простото представяне на числото. **Старшият бит е знаков, а в оставащите битове е записана абсолютната стойност на числото.** Използва се главно за съхраняване на числата в паметта.

$$x_{\text{пр}} = \begin{cases} x, & \text{за } x \geq 0 \text{ за положително число } x_{\text{пр}} = 0, x_1, x_2, \dots x_n \\ 1 + |x|, & \text{за } x < 0 \text{ за отрицателно число } x_{\text{пр}} = 1, x_1, x_2, \dots x_n \end{cases}$$

Пример 1: Числото 3 в прав код се представя в осембитово число като 00000011.

Пример 2: Числото -3 в прав код се представя в осембитово число като 10000011.



БС. Прав, обратен и допълнителен код

Обратен код:

Получава се от правия код на числото, чрез **инвертиране** (заместване на всички нули с единици и единици с нули). Този код не е удобен за извършването на аритметичните действия + и -. Освен това знаковите битове се обработват отделно от информационните. Този недостатък се избягва с употребата на допълнителен код, при който вместо изваждане се извършва събиране с отрицателно число. Последното е представено чрез неговото допълнение т.е. разликата между 2^n и самото число.

$$x_{\text{обр}} = \begin{cases} x, & \text{за } x \geq 0 \\ 2 - 2^{-n} + x, & \text{за } x \leq 0 \end{cases}$$

$$2 - 2^{-n} + x = 1 + 1 - 2^{-n} - |x| = 1.111 \dots 11 - 0.x_1.x_2 \dots x_n$$

Пример 1: Числото 3 в прав код се представя като 0 0000011, а в обратен код има вида 0 111100.

Пример 2: Числото -127 в прав код се представя като 1 1111111, а в обратен код като 1 0000000



БС. Прав, обратен и допълнителен код

Допълнителен код:

Допълнителният код е число в обратен код, към което е прибавена (чрез събиране) единица.

Пример 1:

Числото -127 в прав код се представя като 1 1111111, а в обратен код като 1 0000000.

Числото -127 представено в допълнителен код има вида
1 0000001.



БС. Двоично – десетичен код

Двоично-десетичен код:

Известен е още като **B**CD код (**B**inary **C**oded **D**ecimal).

При този код в един байт се записват по две десетични цифри. Това се постига, като чрез всеки полубайт се кодира една десетична цифра.

Числа представени чрез този код могат да се пакетират т.е. да се представят в пакетирани формат.

Ако представим една десетична цифра в един байт се получава непакетиран формат.



Алгоритми за аритметични действия

Алгоритъм за събиране/изваждане на числа с плаваща запетая:

Численият алгоритъм за произволно число с плаваща запетая е:

$$Nx = \pm M * P^{+-n}$$

Операцията събиране/изваждане на две числа се извършва по следният алгоритъм:

$$A \pm B = M_1 * P^{n1} \pm M_2 * P^{n2} = M_1 \pm M_2 * P^{+-(n1-n2)}$$

първи операнд $A = M_1 * P^{n1}$; втори операнд $B = M_2 * P^{n2}$;

трети операнд **резултат** $C = (M_1 \pm M_2) * P^{n1}$ ако $n1 > n2$

или $C = (M_1 \pm M_2) * P^{n2}$ ако $n1 < n2$

Преди да се приложи алгоритъма е необходимо да се извърши процедура по изравняване на порядъците. Тази процедура може да се алгоритмизира, за което обаче е необходима конкретна програмна среда, защото порядъка се явява част от полето и неговото преобразуване (изравняване) изисква средства за работа на ниво бит или байт.



Алгоритми за аритметични действия

Алгоритми за умножение:

Реализират (схемно или програмно) изчисляването на $Z=X*Y$ за числа с фиксирна или плаваща запетая.

Разглеждаме схемна реализация на числа с фиксирана запетая.

Видове:

- основни;
- ускорени.



Алгоритми за аритметични действия

Алгоритми за умножение. Основни алгоритми:

Основните алгоритми имат **2** варианта:

1-ви вариант – умножението започва с младшия разряд на множителя;

2-ри вариант – умножението започва със старшия разряд на множителя.

И двата варианта има по **два** подварианта:

- Частичното произведение е неподвижно;
- Множимото е неподвижно



Алгоритми за аритметични действия

Алгоритми за умножение. Основни алгоритми:

1. 1-ви вариант – умножението започва с младшия разряд на множителя и частичното произведение е неподвижно, мести се **множимото**:

Пример:

```
0.1101
0.1011
-----
      1101
+   1101
-----
    100111
+  0000
-----
    100111
+  1101
-----
0.10001111
```

Формален алгоритъм:

C1: $Z=0; i=1;$

C2: Ако $Y_n=1$, тогава $Z=Z+X;$

C3: Ако $i < n$, тогава $i=i+1;$

$Y=Y/2; X=2X$ и преход към C2;

Ако $i=n$, следва край.



Алгоритми за аритметични действия

Алгоритми за умножение. Основни алгоритми:

Критерий за оценка на алгоритъм 1-ви вариант:

- а) брой елементи (тригери и др.) с които се реализира;
- б) бързодействие.

X трябва да се съхранява в преместващ регистър с **$2n-1$** разряда, т.к след началното си състояние се мести **$n-1$** пъти. **Y** е преместващ регистър с **n** разряда, а **Z** е суматор с **$2n$** разряда. Следователно, **общия брой разряди е $5n-1$**

Бързодействието е **$T=nT_{\text{съб}}$** .

Алгоритъмът е бърз, но изисква много памет ($5n-1$)



Алгоритми за аритметични действия

Алгоритми за умножение. Основни алгоритми:

2. 2-ри вариант – умножението започва с младшия разряд на множителя, но множимото е неподвижно, а се местят частичните произведения;

При местенето на дясно старшите разреди на **Y** се освобождават и в тях могат да се записват младшите на **Z**.

Така необходимият брой разряди на устройствата стават **$3n$**

Формален алгоритъм:

C1: $Z=0; i=1;$

C2: Ако $Y_n=1$, тогава $Z=Z+X;$

C3: Ако $i < n$, тогава $i=i+1;$

$Y=Y/2; Z=Z/2$ и преход към C2;

Ако $i=n$, следва край.



Алгоритми за аритметични действия

Алгоритми за умножение. Основни алгоритми:

3. 3-ти вариант – умножението започва със старшия разряд на множителя, неподвижни частични произведения, мести се множимото;

$T = nT$ съб., защото събирането ($Z = Z + X$) е по-дългата операция и може да се извърши едновременно с преместването на X ($X = 2X$)

Формален алгоритъм:

C1: $Z = 0; i = 1;$

C2: Ако $Y_1 = 1$, тогава $Z = Z + X;$

C3: Ако $i < n$, тогава $i = i + 1;$

$Y = 2Y; X = X/2$ и преход към C2;

Ако $i = n$, следва край.



Алгоритми за аритметични действия

Алгоритми за умножение. Основни алгоритми:

4. 4-ти вариант – умножението започва със старшия разряд на множителя, множимото е неподвижно, местят се частичните произведения;

Формален алгоритъм:

Общото време е:

$T = n(T_{\text{съб}} + T_{\text{преместване}})$,

защото тук не могат да се съвместят $Z = Z + X$ и $Z = Z/2$

в двете стъпки.

Изисква по-малко памет:

x - n разряда, защото не се мести;

y - n разряда, и z - $2n$ разряда,

т.е. **общо $4n$ разряда**

C1: $Z = 0; i = 1;$

C2: Ако $Y_1 = 1$, тогава $Z = Z + X;$

C3: Ако $i < n$, тогава $i = i + 1;$

$Y = 2Y; Z = Z/2$ и преход към C2;

Ако $i = n$, следва край.



Алгоритми за аритметични действия

Алгоритми за умножение. Ускорено умножение:

5. Алгоритми за ускорено умножение - основават се на два принципа:

а) При алгоритмичните методи се намалява броя на елементарните сумирания и измествания, чрез **едновременно умножаване** с два разреда от множителя.

Тъй като двата разреда на множителя могат да приемат четири различни стойности (00,01,10,11), частичното произведение и неговото сумиране се изпълняват по четири различни начина. Изместванията в регистрите и суматора се извършва едновременно с два разреда, а цикълът за умножение е **$n/2$** . При равномерно разпределение на нулите и единиците в множителя средното време за ускореното по този начин умножение се оценява: **$t = n(t_n + 3/4 \cdot t_c)/2$** .



Алгоритми за аритметични действия

Алгоритми за умножение. Ускорено умножение:

б) При втората група методи (**схемни**) ускоряването на умножението се основава на използването на повече апаратура за пресмятане, като времето за умножение намалява поради едновременното формиране на части от изразите.

Времето за умножение на две **n**- разредни числа се определя единствено от времето **t** за събиране в едноразредния суматор и се оценява по следния начин: $t = (2^n - 1)t_c$.



Алгоритми за аритметични действия

Алгоритми за деление:

1-ви алгоритъм: Ако даден процесор няма деление като операция в своята система от команди, а има инструкции за събиране, изваждане и умножение се използва формулата:

$$z = \frac{x}{y} = x \frac{1}{y}$$

Целта е да се намери начално приближение, така, че общият брой операции (за приближението и итерациите) да е минимален.



Алгоритми за аритметични действия

Алгоритми за деление:

2-ри алгоритъм: За деление с възстановяване на остатъка.

Разглежда се вариант на деление на дробни числа с фиксирана запетая (0 до 1 по абсолютна стойност), от където следва, че **X трябва да е по-малко от Y за да бъде и $Z < 0$** ($Z = X/Y$). В противен случай делението се прекратява.

Формален алгоритъм:

C1: $R = X$; $i = 1$; $R = R - Y$;

C2: Ако $R > 0$, следва **край**. { стъп. C1 и C2 са начало на делението }

C3: $R = R + Y$ { възстановяване на остатъка };

$R = 2R + Y$ { мести се наляво R и се изважда Y };

C4: Ако $R < 0$, то $Z_i = 1$; Ако $R \geq 0$, то $Z_i = 0$ и $R = R + Y$ { възстановяване на остатъка };

C5: Ако $i = n$, следва **край**. Ако $i < n$, тогава $i = i + 1$ и **преход към C4**.



Алгоритми за аритметични действия

Алгоритми за деление:

3-ти алгоритъм: За деление без възстановяване на остатък.

При **отрицателен** остатък вместо възстановяване на последния положителен остатък, той **се мести** наляво и се събира с делителя, т.е. така се избягва възстановяването като се намалява броя на извършените действия. При **положителен** остатък той **се мести наляво и се изважда** делителя.

Формален алгоритъм:

C1: $R=X$; $i=1$; $R=R-Y$;

C2: Ако $R \geq 0$, следва **край**. {стъп. C1 и C2 са начало на делението}

C3: $R=2R+Y$ {места се наляво R и се добавя Y};

C4: Ако $R < 0$, то $Z_i = 0$ и $R=2R+Y$; Ако $R \geq 0$, то $Z_i=1$ и $R=2R-Y$;

C5: Ако $i=n$, следва **край**. Ако $i < n$, тогава $i=i+1$ и **преход към C4**.



Алгоритми за аритметични действия

Алгоритми за деление:

4-ти алгоритъм: За деление на цели числа $Z=X/Y$.

Z (заедно със знака) и X се разглеждат като едно цяло и се местят едновременно.

Формален алгоритъм:

C1: Преместване на 1 разряд наляво на (Z,X) ;

$i=1$; $Z=Z-Y=Z+(-Y)$ доп;

C2: Ако знака на $Z \geq 0$, то $X(0)=1$;

преместване с 1 разряд наляво на (Z,X) ; $Z=Z-Y$;

Ако знака на $Z < 0$, то $X(0)=0$;

преместване с 1 разряд наляво на (Z,X) ; $Z=Z+Y$;

C3: Ако $i=n$, следва край. Ако $i < n$, тогава $i=i+1$ и преход към C2.



Коректност (верификация) на алгоритъм

Верификация на алгоритъм:

Начини за проверка на коректността на алгоритъм:

- Експериментален
- Аналитичен.

1. Експериментален – най-често използван. Използва тестови набори от входни данни и се проверява дали се получават съответните резултати. Този начин не проверява всички участъци от алгоритъма, следователно грешката може да остане там.

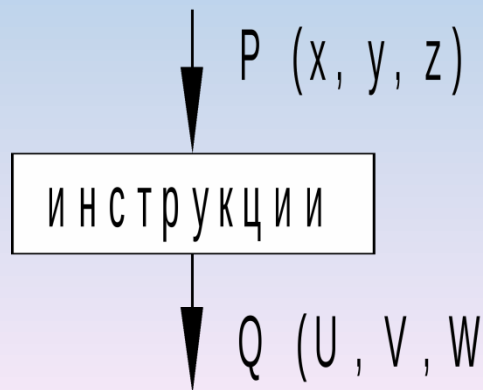
Не гарантира 100% коректност на алгоритъма



Коректност (верификация) на алгоритъм

Аналитичен метод

2. Аналитичен – на входа на алгоритъма се задават входни **условия Р** (покриващи цялата му дефиниционна област) – това са условия, на които се подчиняват входните променливи на алгоритъма. **На изхода се задават изходни условия Q**, на които се подчиняват резултатите (изходните променливи). Във вътрешните точки също могат да се задават междинни условия. **Ако** при преминаване през инструкциите на целия алгоритъм **от входните условия се получат условия, съвпадащи** със зададените изходни условия, тогава алгоритъма е **верен**.



x, y, z – входни променливи

u, v, w – изходни променливи



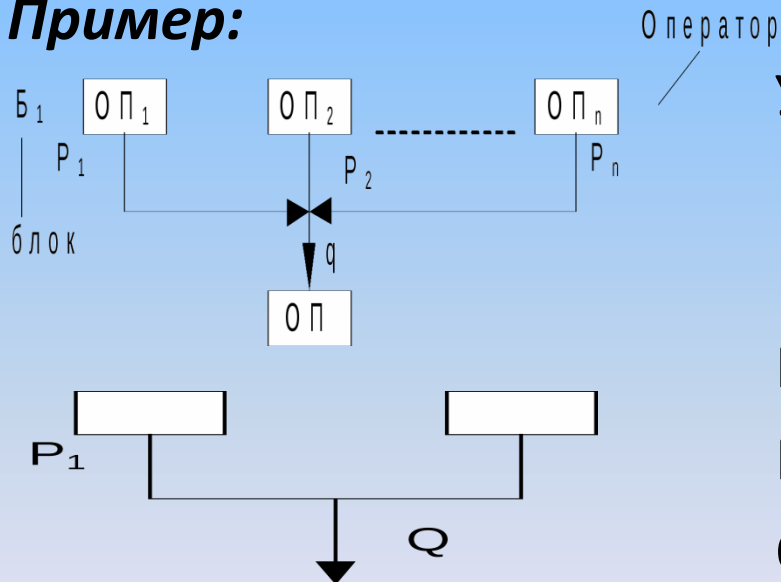
Коректност (верификация) на алгоритъм

Аналитичен метод

а) при обединение на изходите на няколко оператора:

Ако се преобразува входното условие по инструкциите и се получи изходното или пък обратно от изходното се получи входното, то алгоритъмът е коректен.

Пример:



Условието q се получава като логично обединение между $P_1, P_2, \dots, P_n$

$$P_1: -10 X < 0$$

$$P_2: 0 X 10$$

$$\text{От } P_1 \cup P_2 \gg Q \text{ е } -10X+10$$

Може да има застъпване (P_2 да е $-2X10$), но това не променя обединението.



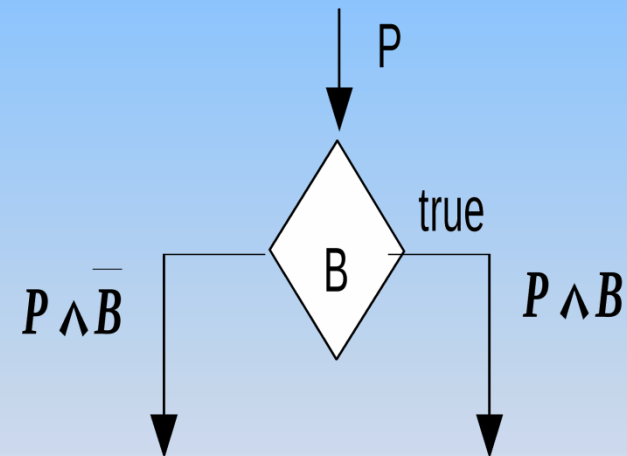
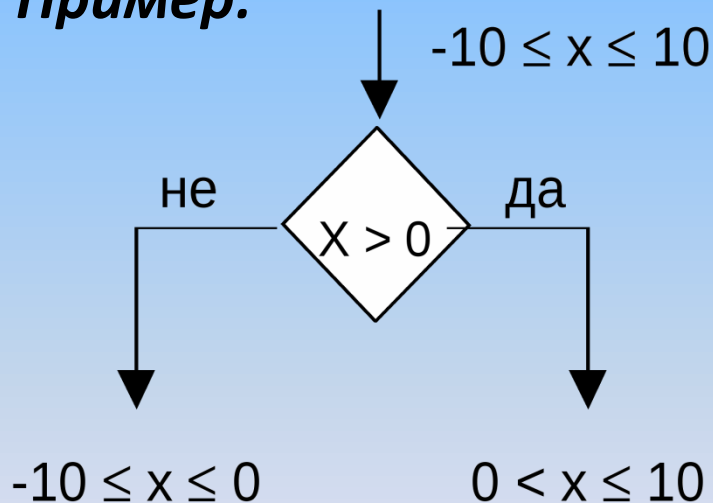
Коректност (верификация) на алгоритъм

Аналитичен метод

б) при разклонение на два клона:

При входни условия P и логически израз B за разклонение имаме изходни условия: $Q1 = P \wedge \bar{B}$ и $Q2 = P \wedge B$ (черта)

Пример:



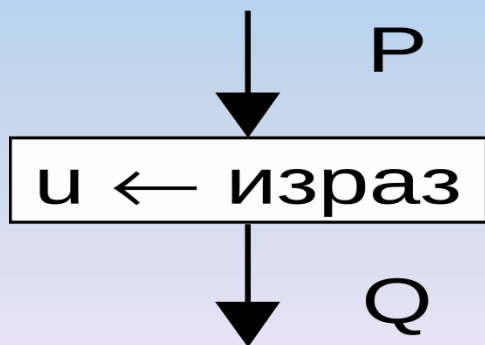


Коректност (верификация) на алгоритъм

Аналитичен метод

в) при оператор за присвояване:

Правилото се нар. **Ефект на присвояване** и гласи: Нека имаме оператор за присвояване **$u=v$** , където **v** е израз, а **u** – променлива. Условието **Q** се получава чрез заместване на всички непресичащи се влягания на израза **v** в **P** с **u** . Ако в **P** няма нито едно влягане на **u** , тогава **P** се преобразува така, че да има поне едно влягане. Обратно, от **Q** може да се получи **P** чрез замяна на всяка поява в **Q** на **u** с **v** .

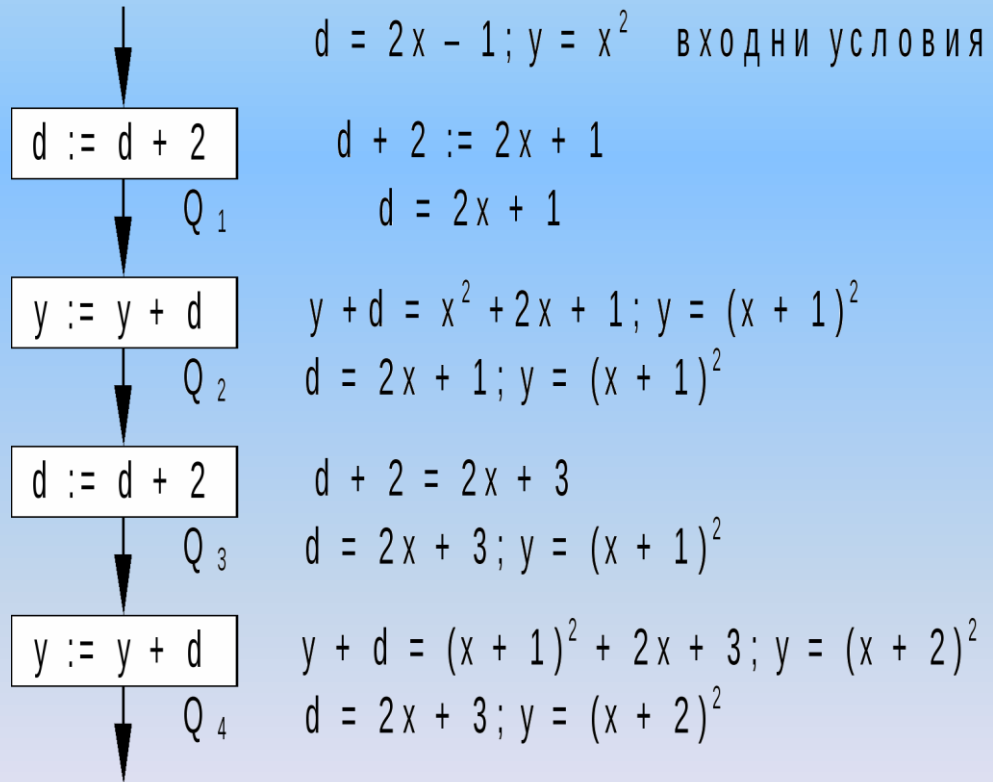
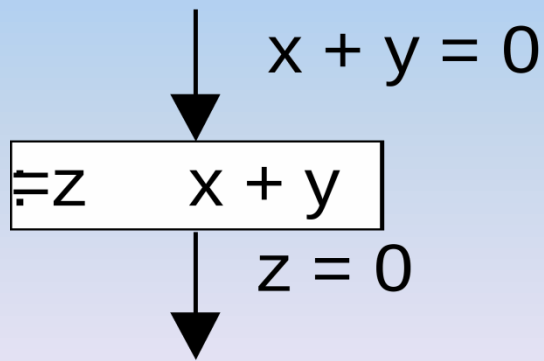
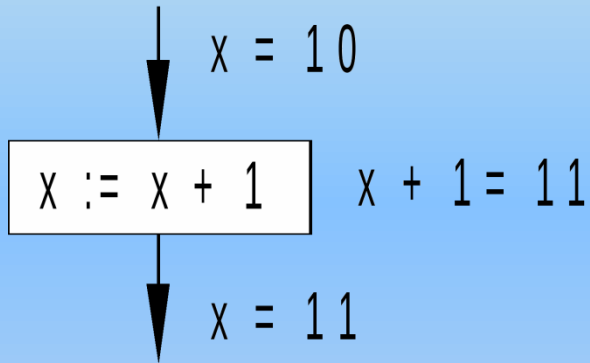




Коректност (верификация) на алгоритъм

Аналитичен метод

Примери:



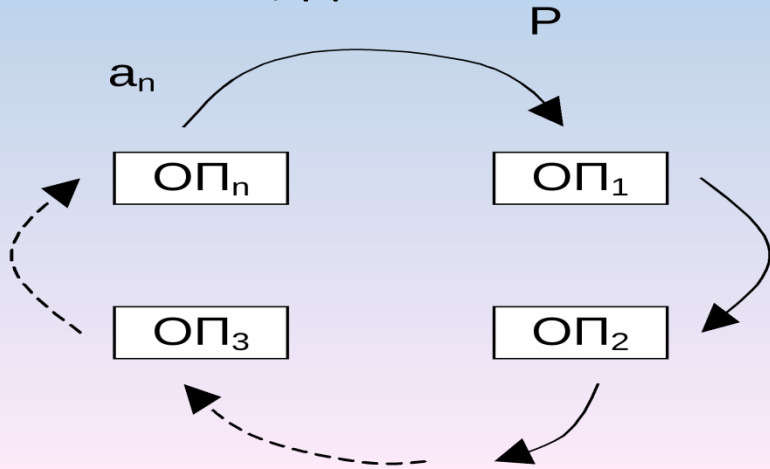
Коректност (верификация) на алгоритъм

Аналитичен метод

г) при оператори за цикъл:

Изобразяваме цикъла като кръгова последователност от оператори, при която след последния оператор $ОП_n$ на цикъла следва първия оператор $ОП_1$. (това се нар. Инвариантно условие)

Условието P ще наречем инварианта, ако $P \rightarrow Q_n$ или ако P следва от Q_m . Ако знаем инвариантата, коректността лесно се доказва, но за съжаление инвариантата не може да се определи аналитично ако можеше, доказването става автоматично.

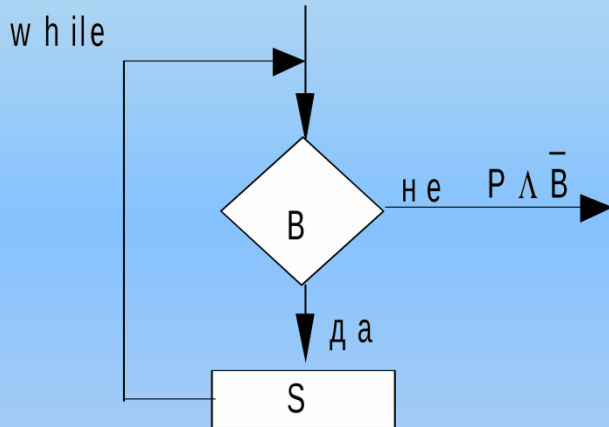




Коректност (верификация) на алгоритъм

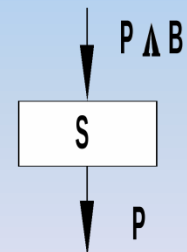
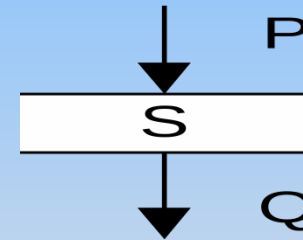
Аналитичен метод

Пример 1: за цикъл while



Може да имаме следните условия:

- От входно условие P на T се получава изходно условие P
- От входно условие $P \wedge B$ на T се получава изходно условие P , следва, че условието в края на цикъла е $P \wedge B$ (черта)



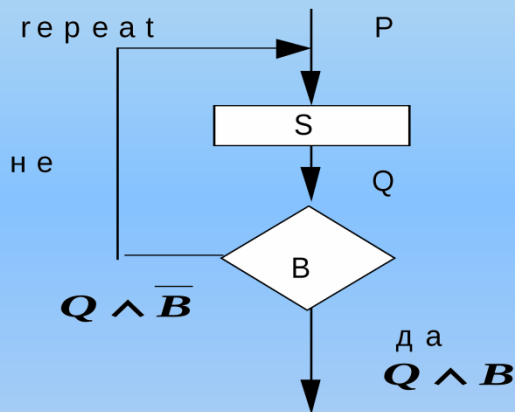
$\Rightarrow$ условието при B за алтернативата не ще е $P \wedge B$



Коректност (верификация) на алгоритъм

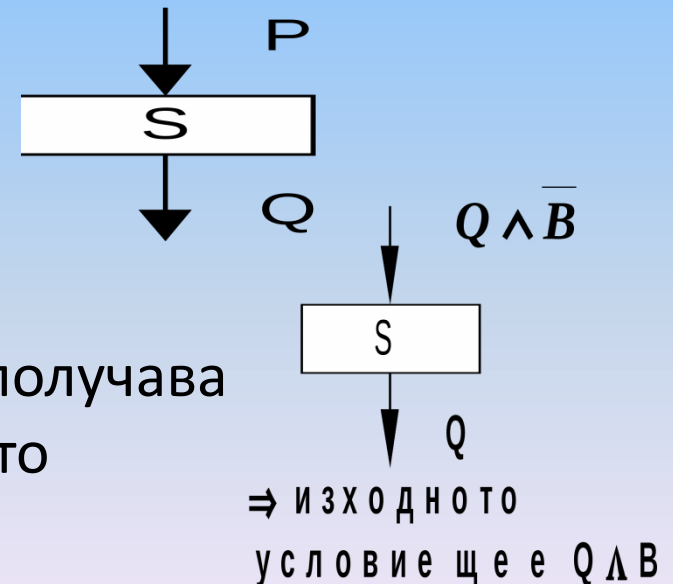
Аналитичен метод

Пример 2: за цикъл repeat



Може да имаме следните условия:

- От входно условие P на T се получава изходно условие Q
- От входно условие $Q \wedge B$ (черта) на T се получава изходно условие Q , следва, че условието в края на цикъла е $Q \wedge B$



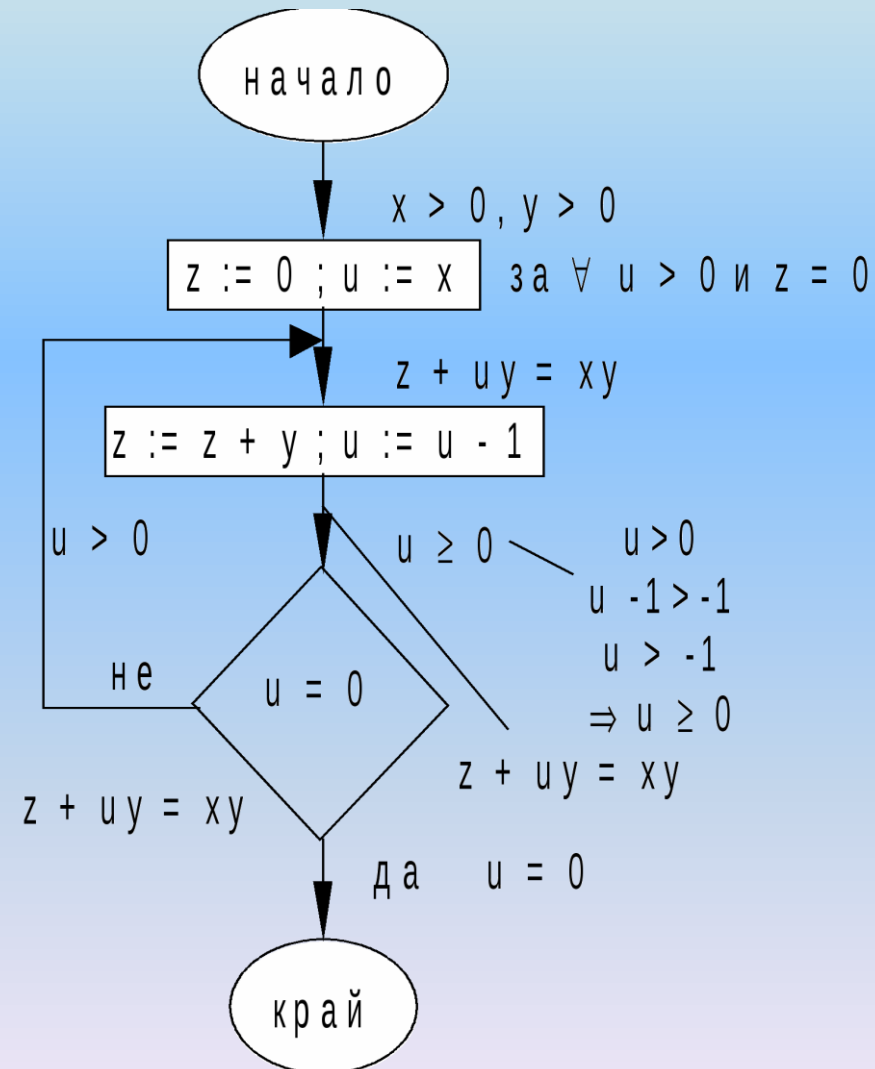
Коректност (верификация) на алгоритъм

Аналитичен метод

Пример 3: верификация на алгоритъма за умножение чрез събиране

Инвариантата е условие, което в дадената точка винаги е вярно. Z го увеличаваме и намаляваме, но винаги в тази точка $Z + uY = XY$. Хитростта е да се измисли точно тази инварианта. Няма правило, по което такава инварианта да се измисли - трябва да се налучка.

Искаме да **докажем, че $Z + uY = XY$** . Използваме ефекта на присвояване. На изхода се получава вярно условие, което сочи, че алгоритъмът е верен.





Коректност (верификация) на алгоритъм

Аналитичен метод

Пример 3: верификация на алгоритъма за сумиране на елементите на едномерен масив

$$S = \sum_{i=1}^n a[i]$$

В началната точка i ще е $i \leq n$, а S в този случай ще е една от сумите на елементите от преди i -тия елемент.

$$S + a[i] = \sum_{j=1}^{i-1} a[j] + a[i] \quad S = \sum_{j=1}^i a[j]$$

На изхода се получава:

$$S = \sum_{j=1}^{i-1} a[j] \quad \text{т.е. Алгоритъма е коректен!}$$

